

触发器和事件

第16章 触发器和事件

- ❑ 触发器（**TRIGGER**）和事件（**EVENT**）都是与表操作相关的特殊类型的存储过程，都包含一系列的控制流**SQL**语句。
- ❑ 触发器（**TRIGGER**）是在满足一定条件下自动触发执行的数据库对象，如向表中插入记录、更新记录或者删除记录时被系统自动地触发并执行。
- ❑ 事件是**MySQL**基于特定时刻或时间周期调用的过程式数据库对象，例如可以在某一时刻定期激活事件向表中插入记录、更新记录或者删除记录，有时也被称作临时触发器。
- ❑ 与触发器不同的是，一个事件可调用一次，也可周期性的调用，它由一个特定的线程来管理，该线程被称作事件调度器（**EVENT SCHEDULER**）。

MySQL 触发器

CONTENTS

This Is A
List

01

触发器概述

02

创建触发器

03

查看触发器

04

删除触发器



01

PART ONE

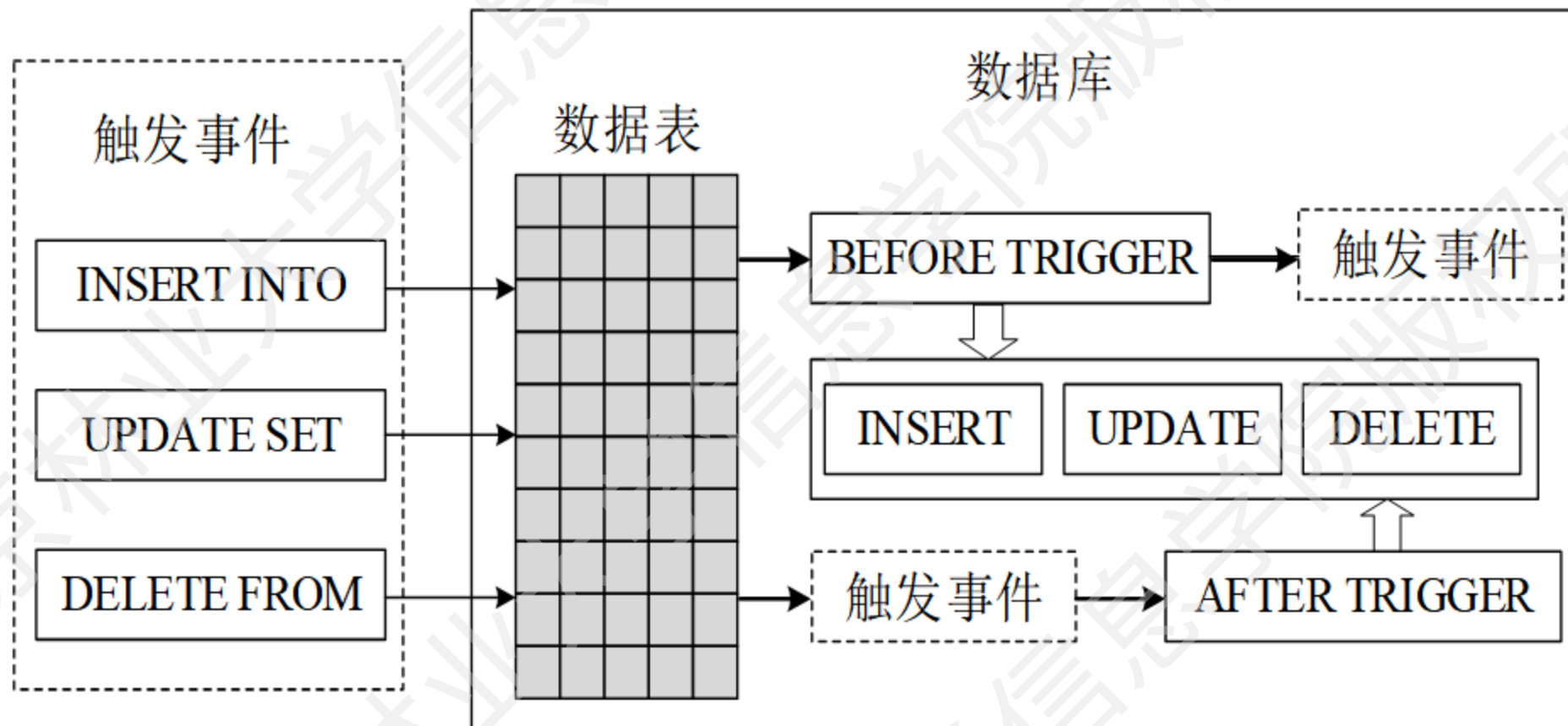
触发器概述

- ❑ 触发器通常是定义在表上的，用于保护表中的数据。触发器针对的是永久性表，而不是临时表，只能由数据库的特定操作事件来触发，当操作影响到保护的数据时触发器与存储过程的区别在于触发器能够自动触发执行。触发器基于一个表创建，但可以基于多个表进行操作，因此，触发器可以对表执行复杂的完整性约束。

触发器

BEFORE触发器：在触发事件之前执行，
用于一些数据的校验工作（数据类型、格式等）

AFTER触发器：在触发事件之后执行，
用于一些后续的统计工作（行数、平均值等）



触发器的分类及执行过程

□ 在MySQL实际应用中，触发器主要可应用于以下场景：

1

数据库的安全性检查

4

数据库的备份和同步

2

数据库的数据校验

5

自动计算数据值

3

数据库的审计

6

实现复杂的数据库完整性规则



02

PART TWO

创建触发器

MySQL触发器操作

01

OPTION

创建触发器

CREATE TRIGGER

02

OPTION

查看触发器

SHOW TRIGGERS,
TRIGGER

03

OPTION

删除触发器

DROP TRIGGER

- MySQL使用CREATE TRIGGER 语句创建触发器，其语法格式为：

```
CREATE[DEFINER={'user'|CURRENT_USER}]
TRIGGER trigger_name trigger_time trigger_event
ON table_name
FOR EACH ROW
[trigger_order] trigger_body
```

- MySQL使用CREATE TRIGGER 语句创建触发器，其语法格式为：

```
CREATE[DEFINER={ 'user' | CURRENT_USER }]  
TRIGGER trigger_name trigger_time trigger_event  
ON table_name  
FOR EACH ROW  
[trigger_order] trigger_body
```

触发程序触发的顺序：指定触发器触发的顺序，按顺序执行。如果触发器触发的顺序相同，则按触发器名称的顺序执行。

创建一个触发器insert_sc_trigger，当sc表中插入某个学生的课程成绩为空时，就把成绩设置为0，如果不是0就按照设定值插入。

(1) 创建触发器insert_sc_trigger的代码如下：

```
DELIMITER $  
CREATE TRIGGER insert_sc_trigger  
BEFORE INSERT  
ON sc  
FOR EACH ROW  
BEGIN  
    IF NEW.score IS NULL THEN  
        SET NEW.score = 0;  
    END IF;  
END$
```

创建一个触发器insert_sc_trigger，当sc表中插入某个学生的课程成绩为空时，就把成绩设置为0，如果不是0就按照设定值插入。

(2) 通过往sc表中插入记录 ('s2','c8',NULL) 验证触发器insert_sc_trigger的功能。

```
INSERT INTO sc VALUES ('s2','c8',NULL);
```

```
SELECT * FROM sc WHERE sno='s2';
```

查询结果如下：

sno	cno	score
s2	苏乐	信息
s2	c5	57.0
s2	c6	81.5
s2	c7	NULL
s2	c4	70.0
s2	c8	0.0

创建一个触发器update_s_trigger，当修改学生表中学生的专业时，把修改时间和修改前后学生的专业信息添加到update_s_log表中作为修改记录。

(1) 在MySQL中，输入以下语句创建一个名为update_s_log的新表。

```
CREATE TABLE update_s_log (  
    sno CHAR(5) NOT NULL,  
    sn VARCHAR(10) NOT NULL,  
    omaj VARCHAR(100) NOT NULL,  
    nmaj VARCHAR(100) NOT NULL,  
    udate DATETIME(0) NOT NULL  
);
```

创建一个触发器update_s_trigger, 当修改学生表中学生的专业时, 把修改时间和修改前后学生的专业信息添加到update_s_log表中作为修改记录。

(2) 创建触发器update_s_trigger的代码如下:

```
DELIMITER $  
CREATE TRIGGER update_s_trigger  
AFTER UPDATE  
ON s  
FOR EACH ROW  
BEGIN  
    INSERT INTO update_s_log VALUES  
    (OLD.sno ,OLD.sn, OLD.maj, NEW.maj,now());  
END$
```

创建一个触发器update_s_trigger，当修改学生表中学生的专业时，把修改时间和修改前后学生的专业信息添加到update_s_log表中作为修改记录。

(3) 验证触发器update_s_trigger的功能，代码及执行结果如下：

```
UPDATE s SET maj = '计算机' WHERE sno = 's2';  
SELECT * FROM update_s_log WHERE sno = 's2';
```

显示结果如下：

sno	sn	omaj	nmaj	update
s2	苏乐	信息	计算机	2021-02-06 16:18:53

创建一个触发器delete_s_trigger，当删除s表中某个人记录时，同时删除sc表中相应的所有课程的成绩。

(1) 创建触发器delete_s_trigger的代码如下：

```
DELIMITER $  
CREATE TRIGGER delete_s_trigger  
BEFORE DELETE  
ON s  
FOR EACH ROW  
BEGIN  
    DELETE FROM sc WHERE sc.sno = OLD.sno;  
END$
```

创建一个触发器delete_s_trigger, 当删除s表中某个人记录时, 同时删除sc表中相应的所有课程的成绩。

(2) 验证触发器delete_s_trigger的功能, 代码如下:

```
DELETE FROM s WHERE s.sno = 's1';  
SELECT * FROM s;  
SELECT * FROM sc;
```



03

PART Three

查看触发器

- ❑ 查看触发器是对数据库中已存在的触发器的定义、状态和语法等信息进行查看。
有以下两种方法来查看触发器的信息。

1.1. SHOW TRIGGERS语句查看触发器

2.2. 通过TRIGGERS表查看触发器

SHOW TRIGGERS语句查看触发器，其语法格式为：

SHOW TRIGGERS \G

通过执行SHOW TRIGGERS命令查看所有触发器的详细信息，包括触发器名称、激活事件、操作对象表、执行的操作等

\G是在DOS环境下来表示触发器信息纵向显示，触发器默认为横向显示，在Workbench中查询时时不需要加\G

✓ 例：查看当前数据库中所有触发器信息



SHOW TRIGGERS \G;

□ 显示结果如下:

```
***** 1. row *****
Trigger: update_s_trigger
Event: UPDATE
Table: s
Statement: BEGIN
INSERT INTO update_s_log VALUES (OLD.sno ,OLD.sn, OLD.maj, NEW.maj,now());
END
Timing: AFTER
Created: 2021-02-06 16:03:29.78
sql_mode: STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION
Definer: root@localhost
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
Database Collation: utf8mb4_0900_ai_ci
***** 2. row *****
Trigger: delete_s_trigger
Event: DELETE
Table: s
Statement: BEGIN
DELETE FROM sc WHERE sc.sno = OLD.sno;
END
```

在s表上存在创建的触发器update_s_trigger，触发事件为UPDATE，触发器执行的操作为BEGIN...END中的语句。触发器执行的顺序为AFTER，其它信息还包括SQL的模式、触发器的定义账户和字符集等。

通过TRIGGERS表查看触发器

在MySQL中，所有的触发器的定义都存储在INFORMATION_SCHEMA数据库下的TRIGGER表中，可以通过SELECT语句来查看所有触发器和特定触发器。

(1) 使用SELECT语句查看所有触发器，其语法格式为：

```
SELECT* FROM INFORMATION_SCHEMA.TRIGGERS;
```

(2) 使用SELECT语句查看特定触发器信息，其语法格式为：

```
SELECT* FROM INFORMATION_SCHEMA.TRIGGERS WHERE condition;
```

✓ 例：查看INFORMATION_SCHEMA中的update_s_trigger触发器信息



```
SELECT*FROM INFORMATION_SCHEMA.TRIGGERS  
WHERE trigger_name='update_s_trigger'\G;
```

□ 显示结果如下:

```
***** 1. row *****
      TRIGGER_CATALOG: def
      TRIGGER_SCHEMA: teaching
      TRIGGER_NAME: update_s_trigger
      EVENT_MANIPULATION: UPDATE
      EVENT_OBJECT_CATALOG: def
      EVENT_OBJECT_SCHEMA: teaching
      EVENT_OBJECT_TABLE: s
      ACTION_ORDER: 1
      ACTION_CONDITION: NULL
      ACTION_STATEMENT: BEGIN
      INSERT INTO update_s_log VALUES (OLD.sno ,OLD.sn, OLD.maj, NEW.maj,now());
      END
      ACTION_ORIENTATION: ROW
      ACTION_TIMING: AFTER
      ACTION_REFERENCE_OLD_TABLE: NULL
      ACTION_REFERENCE_NEW_TABLE: NULL
      ACTION_REFERENCE_OLD_ROW: OLD
      ACTION_REFERENCE_NEW_ROW: NEW

      CREATED: 2021-02-06 16:03:29.78
      SQL_MODE: STRICT_TRANS_TABLES,NO_ENGINE_SUBSTITUTION
      DEFINER: root@localhost
      CHARACTER_SET_CLIENT: utf8mb4
      COLLATION_CONNECTION: utf8mb4_0900_ai_ci
      DATABASE_COLLATION: utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```



04

PART Four

删除触发器

MySQL使用DROP TRIGGER语句来删除已经定义的触发器，其语法格式为：

```
DROP TRIGGER [IF EXISTS] [schema_name.]trigger_name;
```



**SUPER
权限**

 表示语句中必写部分

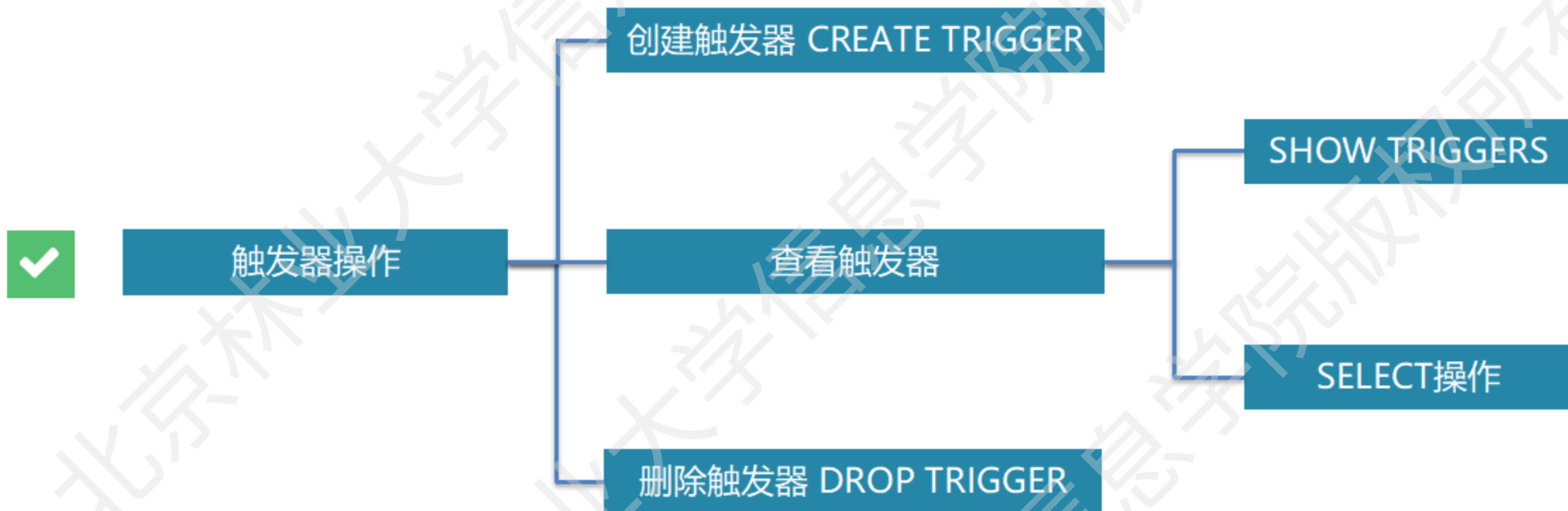
 表示语句中可选部分

执行DROP TRIGGER语句需要SUPER权限。删除一个表的同时，也会自动删除该表上的所有触发器。另外，触发器不能更新或覆盖，为了修改一个触发器，必须先删除它，再重新创建。

- ✓ 例：删除名称为update_s_trigger的触发器。



```
DROP TRIGGER update_s_trigger;
```



MySQL 事件

CONTENTS

This Is A
List

01

事件概述

02

创建事件

03

查看事件

04

修改事件

05

删除事件



01

PART ONE

事件概述

- 事件（**EVENT**）是一种特殊的存储过程，可以用于定时执行的任务，如定时删除记录、对数据进行汇总、清空表、删除表等某些特定任务。事件与触发器类似，都是在某些事务发生时激活。
- 触发器的语句是为了响应给定表上发生的特定类型的操作事件而执行的，而事件的语句是为了响应指定的时间间隔而执行的。因此，有时事件也被称作临时性触发器。

❑ MySQL使用事件调度器（**EVENT SCHEDULER**）来调用事件，它可以不断地监视一个事件是否需要被调用。事件调度器默认是开启的，如果要创建事件，必须打开它。可以通过以下语句开启（**on**）或者关闭（**off**）事件调度器。

- `SET GLOBAL event_scheduler = on|off;`
- `SET @@GLOBAL.event_scheduler = on|off;`
- 在配置文件my.ini（Windows）或者my.cnf（Linux）中加入`event_scheduler=on`，并重启MySQL服务器来开启事件调度器。

❑ 查看事件调度器是否开启

❑ 显示结果如下：

- `SHOW VARIABLES LIKE '%event_scheduler%';`
- `SHOW PROCESSLIST\G;`

Variable_name	Value
event_scheduler	ON



02

PART Two

创建事件

创建事件语句的参数含义

01

OPTION

语句整体结构

CREATE EVENT

02

OPTION

关键参数分析

SCHEDULE

03

OPTION

总结

SUMMARY

- 使用CREATE EVENT语句创建一个事件，其语法格式为：

```
CREATE EVENT [IF NOT EXISTS] event_name
ON SCHEDULE schedule
[ON COMPLETION [NOT] PRESERVE]
[ENABLE|DISABLE|DISABLE ON SLAVE]
[COMMENT 'comment']
DO event_body;
```

表示语句中必写部分

表示语句中可选部分

schedule表示时间调度规则，决定事件激活的时间或者频率。

```
schedule: {AT timestamp[+INTERVAL interval] ...  
|EVERY interval  
[STARTS timestamp [+ INTERVAL interval] ...]  
[ENDS timestamp [+ INTERVAL interval] ...] }
```

AT子句	定义事件发生的时刻
EVERY子句	定义事件在时间区间内每隔多长时间激活事件一次
STARTS子句	指定事件执行的开始时间
ENDS子句	指定事件执行的结束时间

例题1：创建立刻执行的事件event_name1，该事件在数据库teaching中创建一个表tb_one，表的字段包括tno和ttime，tno为主键，插入数据时自动增加，ttime格式为TIMESTAMP

(1) 创建事件event_name1的代码如下：

```
CREATE EVENT event_name1
```

```
ON SCHEDULE AT now()
```

```
DO
```

```
CREATE TABLE tb_one (tno INT PRIMARY KEY
```

```
AUTO_INCREMENT, ttime TIMESTAMP);
```

例题1：创建立刻执行的事件event_name1，该事件在数据库teaching中创建一个表tb_one，表的字段包括tno和ttime，tno为主键，插入数据时自动增加，ttime格式为TIMESTAMP

(2) 验证事件event_name1的功能，代码如下所示：



SHOW tables;

显示结果如下：

Tables_in_teaching
c
s
sc
t
tb_one
tc

例题2：创建立刻执行的事件event_name2，该事件的任务为10秒后往tb_one表中插入一条数据

(1) 创建事件event_name2的代码如下：

```
CREATE EVENT event_name2  
ON SCHEDULE AT CURRENT_TIMESTAMP + INTERVAL 10 SECOND  
DO  
INSERT INTO tb_one VALUES(0, now());
```

AT定义事件发生的时刻，timestamp表示一个具体的时刻，加上一个时间间隔interval，表示在10s后激活事件。

例题2：创建立刻执行的事件event_name2，该事件的任务为10秒后往tb_one表中插入一条数据

(2) 查询事件event_name2的功能，代码如下所示：

```
SELECT * FROM tb_one;
```

显示结果如下：

tno	ttime
1	2021-02-08 16:14:28

例题3：创建立刻执行的事件event_name3，该事件的任务为在时刻2021-02-08 16:29:59往tb_one表中插入一条数据。

(1) 创建事件event_name3的代码如下：

```
CREATE EVENT event_name3  
ON SCHEDULE AT '2021-02-08 16:29:59'  
DO  
INSERT INTO tb_one VALUES(0, now());
```

例题3：创建立刻执行的事件event_name3，该事件的任务为在时刻2021-02-08 16:29:59往tb_one表中插入一条数据。

(2) 查询事件event_name3的功能，代码如下所示：

```
SELECT * FROM tb_one;
```

显示结果如下：

tno	ttime
1	2021-02-08 16:14:28
2	2021-02-08 16:29:59

例题4：创建周期性执行的事件event_name4，该事件的任务为每5秒往tb_one表中插入一条数据。

(1) 创建事件event_name4的代码如下：

```
CREATE EVENT event_name4  
ON SCHEDULE EVERY 5 SECOND  
DO  
INSERT INTO tb_one VALUES(0, now());
```

定义事件在时间区间内每隔5秒被激活事件一次

例题4：创建周期性执行的事件event_name4，该事件的任务为每5秒往tb_one表中插入一条数据。

(2) 查询事件event_name4的功能，代码如下所示：

```
SELECT * FROM tb_one;
```

显示结果如下：

tno	ttime
1	2021-02-08 16:14:28
2	2021-02-08 16:29:59
3	2021-02-08 17:04:44
4	2021-02-08 17:04:49
...	...

例题5：创建周期性执行的事件event_name5，该事件的任务为 '2021-02-08 17:10:00'开始在未来一周内每1小时清空一次tb_one表。

创建事件event_name5的代码如下：

```
CREATE EVENT event_name5
ON SCHEDULE EVERY 1 HOUR
STARTS '2021-02-08 17:10:00'
ENDS CURRENT_DATE + INTERVAL 1 WEEK
DO
DELETE FROM tb_one;
```

例题6：创建事件event_name6调用存储过程pro_score，实现每周查看一下每个学生所有课程的平均分和总分并把查询结果插入到week_score表中，表中的字段包括学生学号（sno）、平均分（avgscore）、总分（sumscore）和插入数据的时间（updatetime）。

(1) 创建表week_score的代码如下：

```
CREATE TABLE week_score (  
    sno char(5) NOT NULL,  
    avgscore INT,  
    sumscore INT,  
    updatetime TIMESTAMP  
);
```

例题6：创建事件event_name6调用存储过程pro_score，实现每周查看一下每个学生所有课程的平均分和总分并把查询结果插入到week_score表中，表中的字段包括学生学号（sno）、平均分（avgscore）、总分（sumscore）和插入数据的时间（updatetime）。

(2) 创建存储过程pro_score，实现查询每个学生所有课程的平均分和总分并把查询结果插入到week_score表中，代码如下：

```
CREATE PROCEDURE pro_score()  
BEGIN  
INSERT INTO week_score SELECT sno, avg(score), sum(score) , now()  
FROM sc GROUP BY sno;  
END;
```

例题6：创建事件event_name6调用存储过程pro_score，实现每周查看一下每个学生所有课程的平均分和总分并把查询结果插入到week_score表中，表中的字段包括学生学号（sno）、平均分（avgscore）、总分（sumscore）和插入数据的时间（updatetime）。

(3) 创建事件event_name6调用存储过程pro_score，代码如下：

```
CREATE EVENT event_name6
ON SCHEDULE EVERY 1 WEEK
DO
BEGIN
    CALL pro_score();
END;
```

例题6：创建事件event_name6调用存储过程pro_score，实现每周查看一下每个学生所有课程的平均分和总分并把查询结果插入到week_score表中，表中的字段包括学生学号（sno）、平均分（avgscore）、总分（sumscore）和插入数据的时间（updatetime）。

(4) 验证事件event_name6的功能，代码如下：

```
SELECT * FROM week_score;
```



03

PART Three

查看事件

❑ 在当前数据库下，当创建好事件后，用户可以通过以下方式查询事件的信息。

1.1. 通过SHOW EVENTS语句查询当前库中所有事件的信息

- SHOW EVENTS;

2.2. 通过SHOW CREATE EVENT语句查询特定事件的信息

- SHOW CREATE EVENT event_name;

- 显示当前数据库下所有事件的信息。

```
SHOW EVENTS \G
```

通过执行SHOW EVENTS命令查看所有事件的详细信息

***** 1. row *****

Db: teaching
Name: event_name5
Definer: root@localhost
Time zone: SYSTEM
Type: RECURRING
Execute at: NULL
Interval value: 1
Interval field: HOUR
Starts: 2021-02-08 17:10:00
Ends: 2021-02-16 00:00:00
Status: ENABLED
Originator: 1
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
Database Collation: utf8mb4_0900_ai_ci

***** 2. row *****

Db: teaching
Name: event_name7
Definer: root@localhost
Time zone: SYSTEM
Type: RECURRING
Execute at: NULL
Interval value: 5
Interval field: SECOND
Starts: 2021-02-09 10:18:36
Ends: NULL
Status: ENABLED
Originator: 1
character_set_client: utf8mb4
collation_connection: utf8mb4_0900_ai_ci
Database Collation: utf8mb4_0900_ai_ci
2 rows in set (0.03 sec)

- 显示当前数据库下事件名称为event_name5的信息。

```
SHOW CREATE EVENT event_name5 \G;
```

通过执行SHOW CREATE EVENT 命令查看特定事件的详细信息



04

PART Four

修改事件

- ❑ 在MySQL中，可以使用ALTER EVENT语句修改事件的定义和相关属性，如事件的名称、状态、注释等，语法格式如下所示：

```
ALTER EVENT [IF NOT EXISTS] event_name  
[ON SCHEDULE schedule]  
[ON COMPLETION [NOT] PRESERVE]  
[RENAME TO new_event_name]  
[ENABLE|DISABLE|DISABLE ON SLAVE]  
[COMMENT 'comment']  
[DO event_body];
```

当使用ON COMPLETION [NOT] PRESERVE属性定义的事件最后一次执行后，事件就不存在了，因此，也不需要再修改事件了。

—— 表示语句中必写部分

例：事件event_name4进行操作如下：将每5秒插入一条数据，修改成每分钟插入一条数据，将事件的名称改为name4并对事件添加注释“添加测试用例的注释”。

```
ALTER EVENT event_name4 ON SCHEDULE EVERY  
1 MINUTE  
RENAME TO name4  
COMMENT '添加测试用例的注释';
```

对事件event_name5修改为当事件到期了，事件就被自动删除。

```
ALTER EVENT event_name5 ON COMPLETION NOT PRESERVE;
```



05

PART Five

删除事件

❑ 在MySQL中，使用**DROP EVENT**删除事件，语法格式如下所示：

```
DROP EVENT [IF NOT EXISTS] event_name;
```

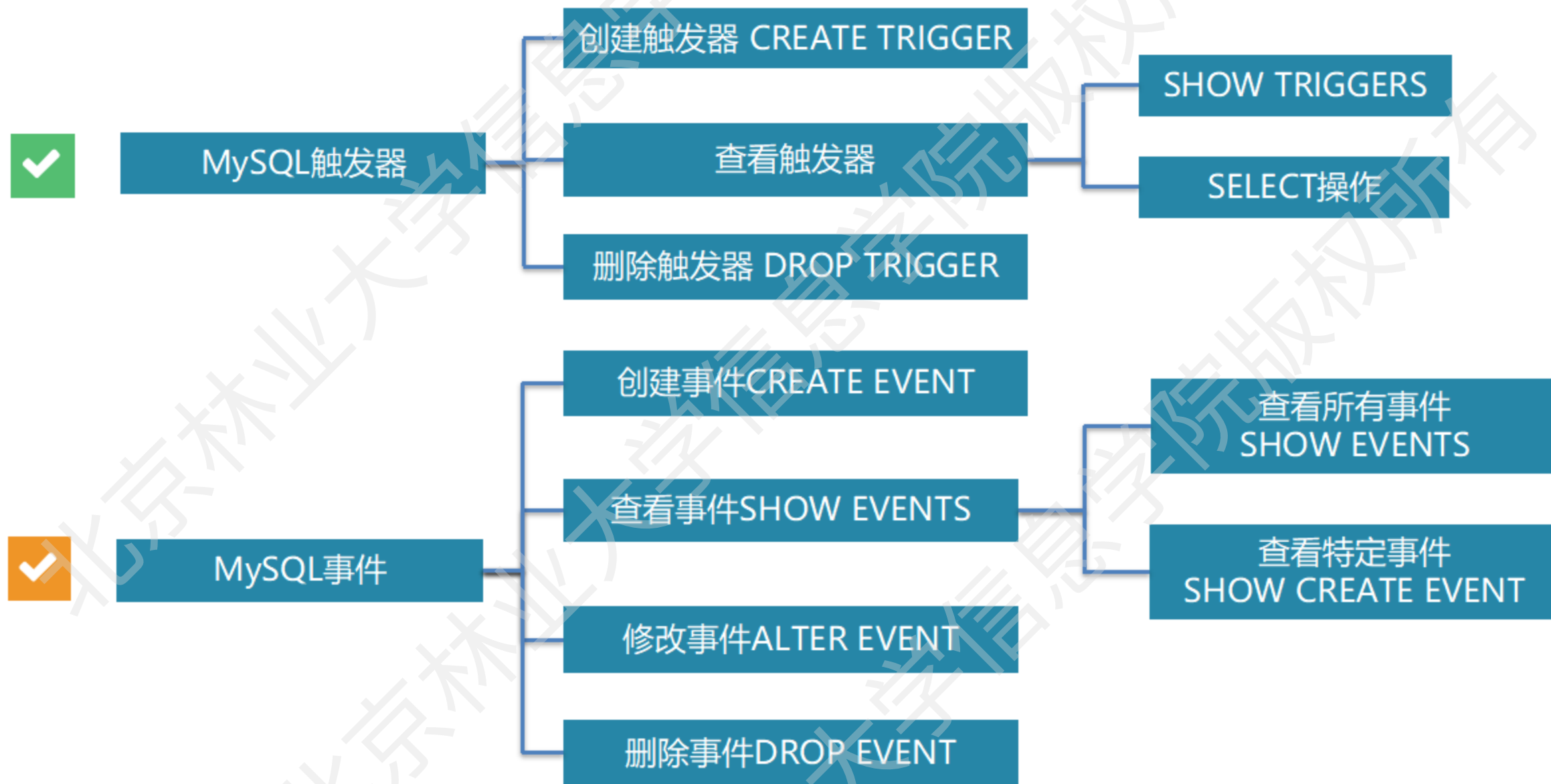
—— 通过执行DROP EVENT命令删除特定的事件

例：从当前数据库删除名称为event_name5的事件。

```
DROP EVENT event_name5;
```



总结





人邮教育

www.ryjiaoyu.com

更多样书申请和资源下载需求，请登录人邮教育社区
(www.ryjiaoyu.com)

海量图书方便查询

囊括各大品类，您想要的应有尽有

免费申请样书

教师免费申请样书，
我们将安排快递迅速送达

下载配套资源

教学视频、PPT课件、教学案例、
习题答案、模拟试卷等丰富资源
免费下载



优惠购书

教师可以申请最低折扣
学生直接优惠购买图书

成为作者

欢迎写文章 / 投稿，我们强大的编辑团队将为您提供专业和高效的编辑出版服务